



INTERRUPTS



WHO AM I

LOAY AHMED MOHMED

STUDYING IN CUFE
EPM (SECOND YEAR)
AN IEEE'S EMBEDDED INSTRUCTOR

MY LINKEDIN ->>>>>



01014751158



loay.ahmed30h@gmail.com



WHAT IS AN INTERRUPT

An Interrupt is a signal that temporarily stops the main program to handle an important event then the program returns to where it stopped

EXAMPLE EVENTS :

- Button press
- Timer overflow
- Data received from UART
- ADC conversion finished



Advantages :

- Faster response
- Saves CPU time
- Efficient programming

INTERRUPT SERVICE ROUTINE (ISR)

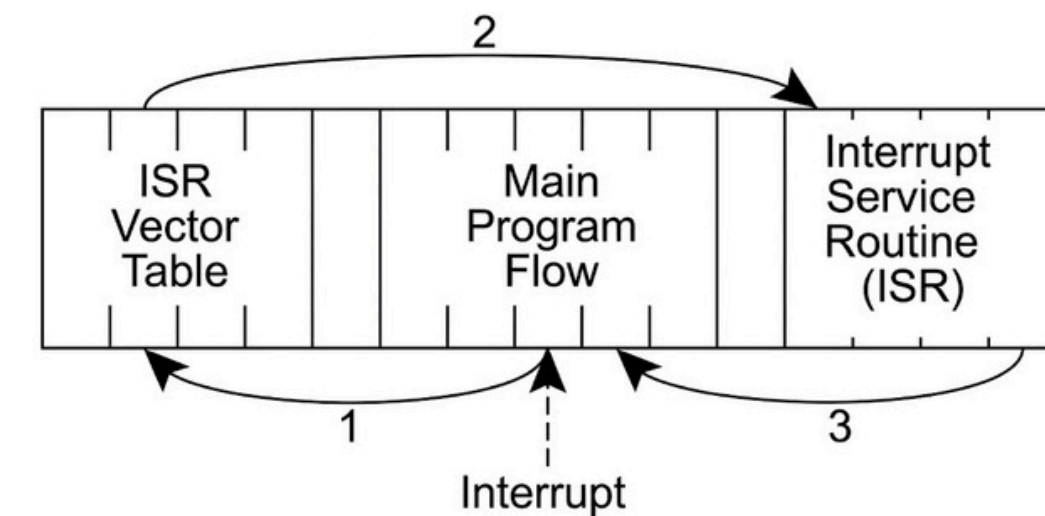
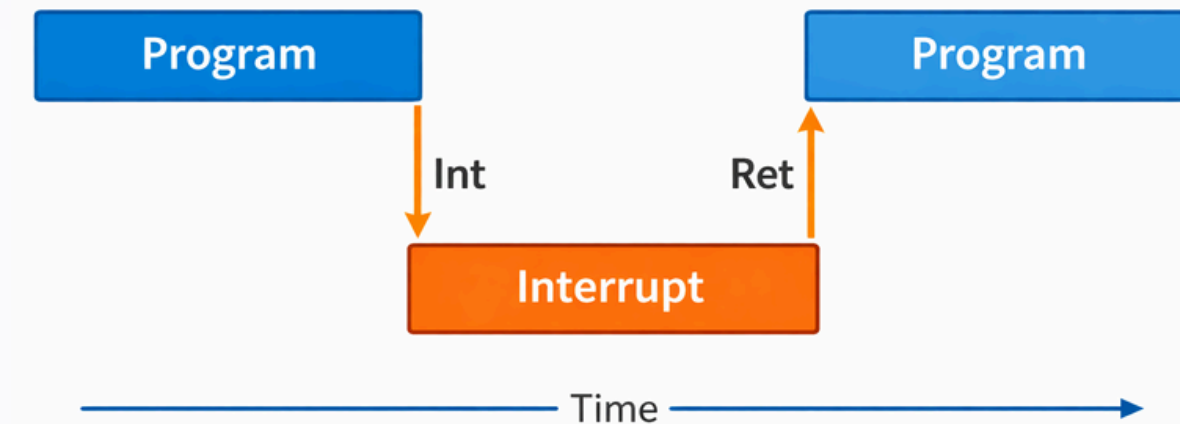
Interrupt Service Routine (ISR) or Interrupt Handler

- A piece of code executed when an interrupt occurs.
- For every interrupt there must be a program associated with it.
- When an interrupt occurs, the CPU runs the ISR.
- Usually each interrupt has its own ISR.
- Its address in ROM is stored in the Interrupt Vector Table.

Characteristics of ISR

ISR should be:

- Deterministic
- Short
- Executed as fast as possible



INTERRUPT SERVICE ROUTINE (ISR)

Guidelines:

- The interrupt should only perform the needed task.
- Return quickly to the main program.
- Avoid loops or long calculations.

Other notes:

- ISR does not have a prototype.
- ISR does not take arguments.
- ISR does not return values.
- It is automatically called by hardware.
- It can use shared variables with the main program.

1. AVR-GCC compiler needs a library to understand ISR definitions.

#include <avr/interrupt.h>

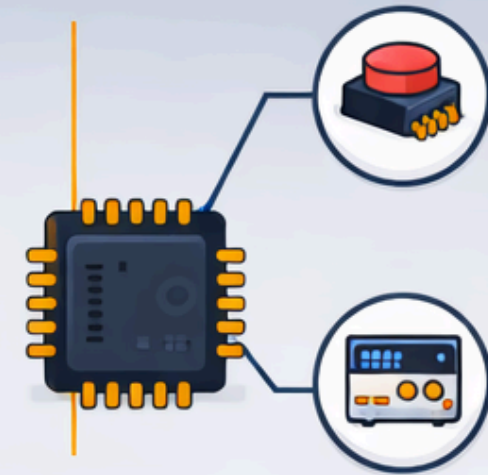
2. Write ISR for a specific interrupt using AVR library:

```
ISR(INTERRUPT_NAME_vect)
{
    /* ISR code */
}
```

Interrupts **Types**

Maskable Interrupts

- It could be **disabled** or **enabled** depends on the **Global Interrupt Enable** bit inside the processor.
- In AVR Processor this bit is called I-bit and it is located in the **Processor Status Register (SREG)**.
- For example :
 -  External Interrupt event on I/O Pin.
 -  IRQs from peripherals in the microcontroller.



Non-Maskable Interrupts

- Interrupts **cannot be disabled** and are **always enabled**.
- It **cannot be masked by software**.
- **NMI** is always assigned with **highest interrupt priority level**. That's why it is commonly used in real time applications.
- AVR processor only support **one NMI** which is **reset interrupt**.



Interrupt Generation Conditions

For an interrupt to occur, **three** main conditions must be satisfied:

1 Global Interrupt Enable (GIE / I-bit)

- ▶ The **Global Interrupt Enable** bit must be enabled.
- ▶ In AVR it is called **I-bit**.
- ▶ It is located in the **Status Register (SREG)**.

If this bit = 0 → No interruptions allowed

If this bit = 1 → Interrupts are allowed

2 Interrupt Enable Bit (MIE)

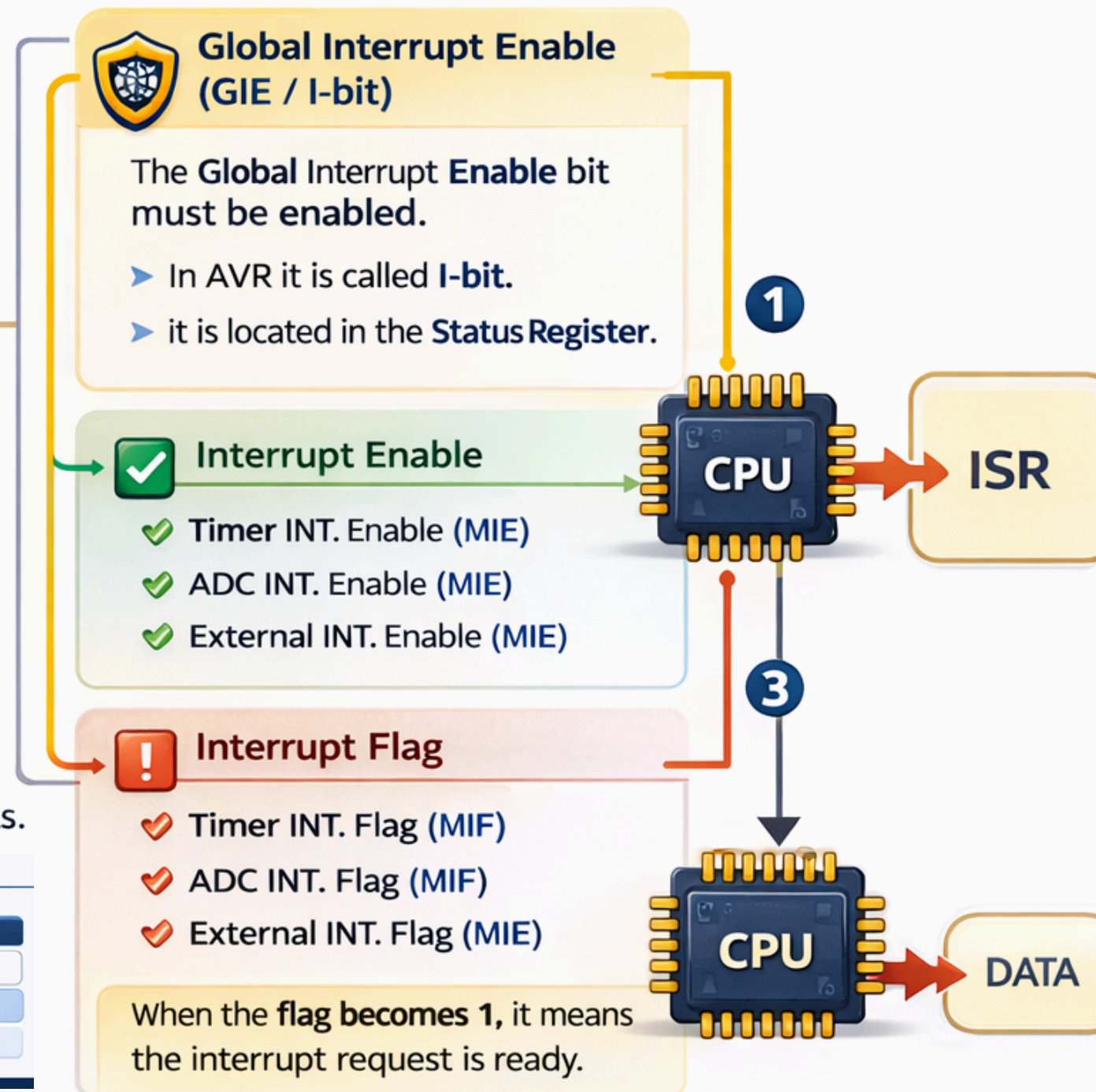
- ▶ Each peripheral has its own Interrupt Enable bit.

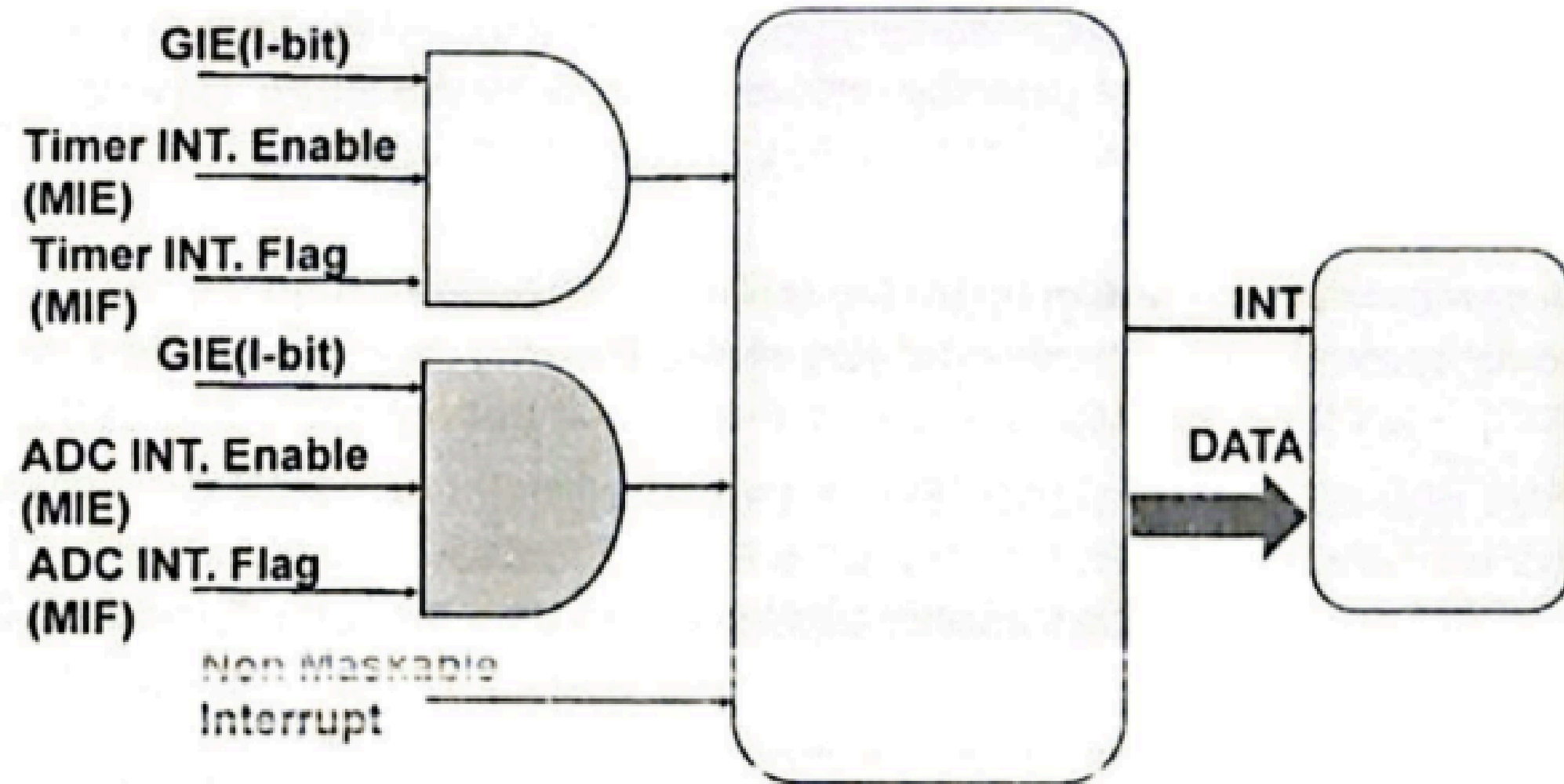
Example:

- ✓ Timer Interrupt Enable
- ✓ ADC INT. Enable (MIE)
- ✓ External INT. Enable (MIE)

- ▶ This bit allows that specific peripheral to generate interrupts.

Processor Status Register (SREG)								
Bit	7	6	5	4	3	2	1	0
Read/Write	I	T	H	S	V	N	Z	C
Initial Value	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	0	0	0	0	0	0	0	0





INTERRUPT VECTOR TABLE (IVT)

- It is a constant table in ROM (Program Memory).
- In most cases, it exists at the beginning of the memory.
- Contains special addresses with respect to the CPU.
- Each interrupt has a specific location in the IVT for its ISR (Interrupt Service Routine).
- This specific location should be programmed to have the address of the ISR for this interrupt.
- ATmega16/32 Microcontrollers: The Interrupt Vector Table has 21 different locations.

VECTOR Table

Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
1	\$000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset, and JTAG AVR Reset
2	\$002	INT0	External Interrupt Request 0
3	\$004	INT1	External Interrupt Request 1
4	\$006	TIMER2 COMP	Timer/Counter2 Compare Match
5	\$008	TIMER2 OVF	Timer/Counter2 Overflow
6	\$00A	TIMER1 CAPT	Timer/Counter1 Capture Event
7	\$00C	TIMER1 COMPA	Timer/Counter1 Compare Match A
8	\$00E	TIMER1 COMPB	Timer/Counter1 Compare Match B
9	\$010	TIMER1 OVF	Timer/Counter1 Overflow
10	\$012	TIMER0 OVF	Timer/Counter0 Overflow
11	\$014	SPI, STC	Serial Transfer Complete
12	\$016	USART, RXC	USART, Rx Complete
13	\$018	USART, UDRE	USART Data Register Empty
14	\$01A	USART, TXC	USART, Tx Complete
15	\$01C	ADC	ADC Conversion Complete
16	\$01E	EE_RDY	EEPROM Ready
17	\$020	ANA_COMP	Analog Comparator
18	\$022	TWI	Two-wire Serial Interface
19	\$024	INT2	External Interrupt Request 2
20	\$026	TIMER0 COMP	Timer/Counter0 Compare Match
21	\$028	SPM_RDY	Store Program Memory Ready

INTERRUPT SOURCES IN AVR

AVR microcontrollers provide both internal and external interrupt sources.

Internal Interrupts:

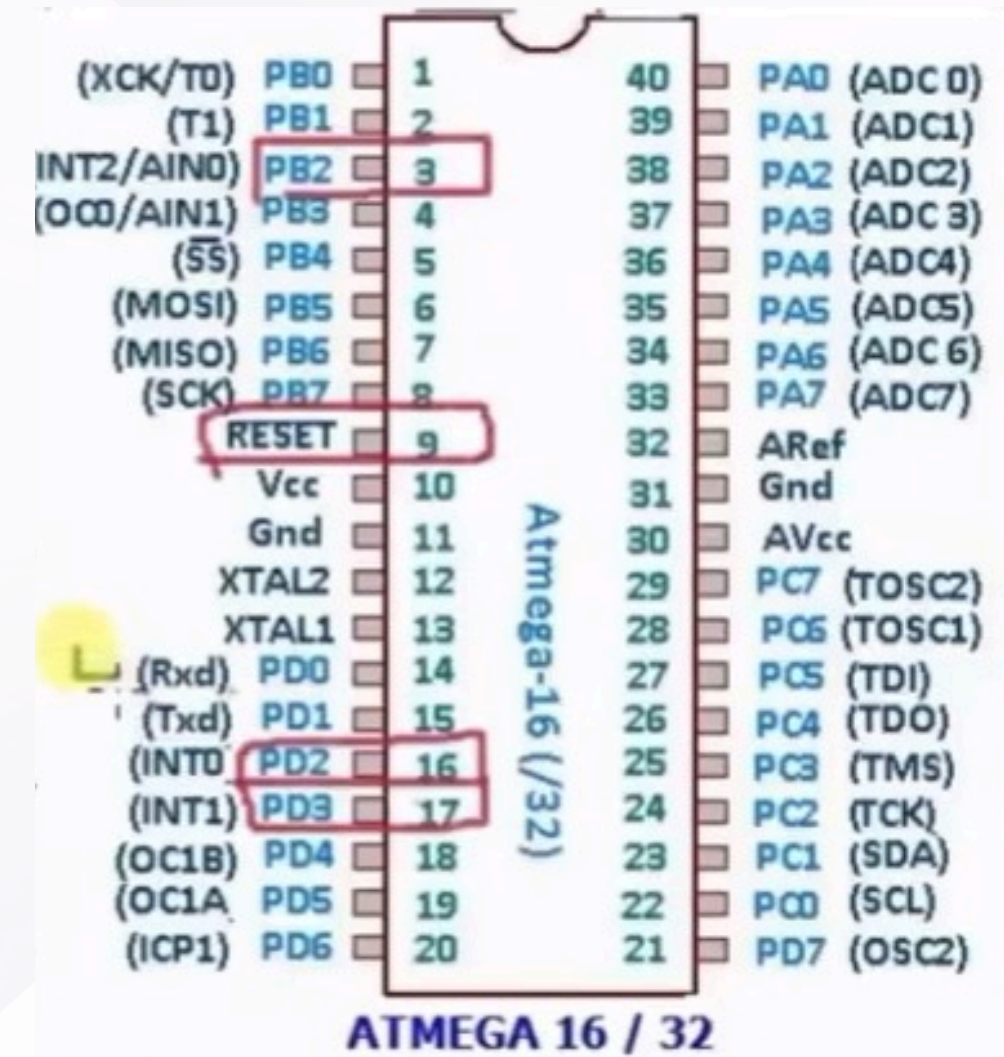
Associated with the microcontroller's peripherals, such as:

- Timer/Counter
- Analog Comparator
- Analog to Digital Converter (ADC)
- USART, etc.
- External Interrupts: Triggered via specific external pins.

ATmega16/32 External Interrupts

For ATmega16/32 microcontrollers, there are four (4) external interrupts:

- RESET interrupt: Triggered from pin 9.
- External Interrupt 0 (INT0): Triggered from pin 16 (PD2).
- External Interrupt 1 (INT1): Triggered from pin 17 (PD3).
- External Interrupt 2 (INT2): Triggered from pin 3 (PB2).



REGISTERS FOR EXTERNAL

MCU Control Register – MCUCR

The MCU Control Register contains control bits for interrupt sense control and general MCU functions.

Bit	7	6	5	4	3	2	1	0	
	SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 3, 2 – ISC11, ISC10: Interrupt Sense Control 1 Bit 1 and Bit 0**

The External Interrupt 1 is activated by the external pin INT1 if the SREG I-bit and the corresponding interrupt mask in the GICR are set. The level and edges on the external INT1 pin that activate the interrupt are defined in Table 34. The value on the INT1 pin is sampled before detecting edges. If edge or toggle interrupt is selected, pulses that last longer than one clock period will generate an interrupt. Shorter pulses are not guaranteed to generate an interrupt. If low level interrupt is selected, the low level must be held until the completion of the currently executing instruction to generate an interrupt.

Table 34. Interrupt 1 Sense Control

ISC11	ISC10	Description
0	0	The low level of INT1 generates an interrupt request.
0	1	Any logical change on INT1 generates an interrupt request.
1	0	The falling edge of INT1 generates an interrupt request.
1	1	The rising edge of INT1 generates an interrupt request.

REGISTERS FOR EXTERNAL

- **Bit 1, 0 – ISC01, ISC00: Interrupt Sense Control 0 Bit 1 and Bit 0**

The External Interrupt 0 is activated by the external pin INT0 if the SREG I-flag and the corresponding interrupt mask are set. The level and edges on the external INT0 pin that activate the interrupt are defined in Table 35. The value on the INT0 pin is sampled before detecting edges. If edge or toggle interrupt is selected, pulses that last longer than one clock period will generate an interrupt. Shorter pulses are not guaranteed to generate an interrupt. If low level interrupt is selected, the low level must be held until the completion of the currently executing instruction to generate an interrupt.

Table 35. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

REGISTERS FOR EXTERNAL

General Interrupt Flag Register – GIFR

Bit	7	6	5	4	3	2	1	0	
	INTF1	INTF0	INTF2	–	–	–	–	–	GIFR
Read/Write	R/W	R/W	R/W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – INTF1: External Interrupt Flag 1**

When an edge or logic change on the INT1 pin triggers an interrupt request, INTF1 becomes set (one). If the I-bit in SREG and the INT1 bit in GICR are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it. This flag is always cleared when INT1 is configured as a level interrupt.

- **Bit 6 – INTF0: External Interrupt Flag 0**

When an edge or logic change on the INT0 pin triggers an interrupt request, INTF0 becomes set (one). If the I-bit in SREG and the INT0 bit in GICR are set (one), the MCU will jump to the corresponding interrupt vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it. This flag is always cleared when INT0 is configured as a level interrupt.

- **Bit 5 – INTF2: External Interrupt Flag 2**

When an event on the INT2 pin triggers an interrupt request, INTF2 becomes set (one). If the I-bit in SREG and the INT2 bit in GICR are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it. Note that when entering some sleep modes with the INT2 interrupt disabled, the input buffer on this pin will be disabled. This may cause a logic change in internal signals which will set the INTF2 Flag. See "Digital Input Enable and Sleep Modes" on page 51 for more information.

CODED EXAMPLE

Practical Task: LED Toggling via External Interrupt

Objective:

Control an LED state (Toggle) using a Push Button connected to one of the External Interrupt pins in ATmega32.

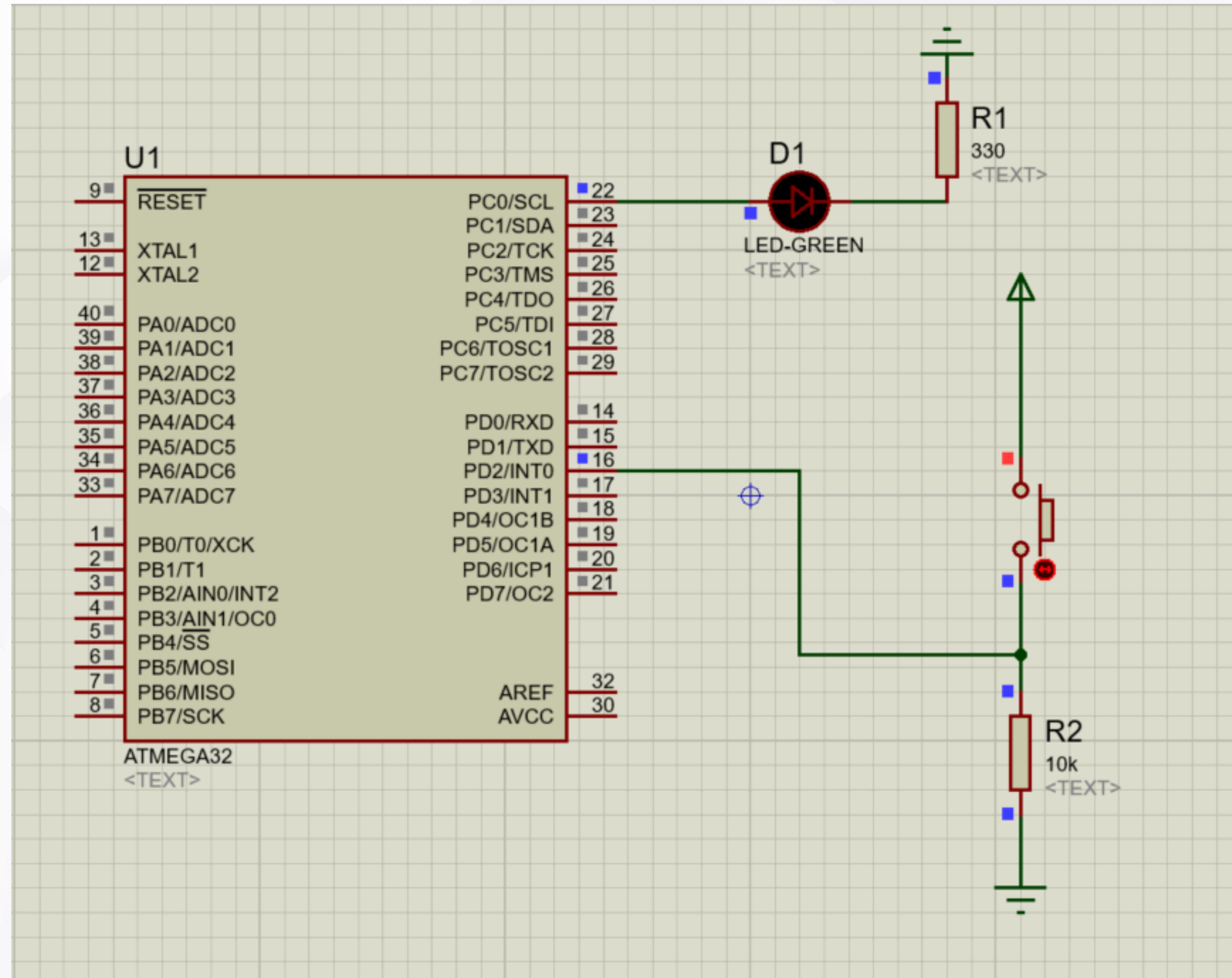
Hardware Connections:

- LED: Connected to PC0 with positive logic
- Push Button: Connected to INTO (Pin 16 / PD2).
- Button Configuration: Use an external Pull-down resistor.

Task Requirements:

- Configure INTO to trigger on a rising Edge (when the button is pressed).
- Write an ISR (Interrupt Service Routine) for INTO.
- Inside the ISR, write the code to Toggle the LED state.
- The Main Loop should remain empty or perform another independent task

CODED EXAMPLE



CODED EXAMPLE

```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 /* External INT0 Interrupt Service Routine */
5 ISR(INT0_vect)
6 {
7     PORTC = PORTC ^ (1<<PC0); //Toggle value of PIN 0 in PORTC (Led Toggle)
8 }
9
10 /* External INT0 enable and configuration function */
11 void INT0_Init(void)
12 {
13     DDRD  &= ~(1<<PD2);           // Configure INT0/PD2 as input pin
14     MCUCR |= (1<<ISC00) | (1<<ISC01); // Trigger INT0 with the raising edge
15     GICR  |= (1<<INT0);           // Enable external interrupt pin INT0
16     SREG  |= (1<<7);             // Enable interrupts by setting I-bit
17 }
18
19 int main(void)
20 {
21
22     INT0_Init();                 // Enable external INT0
23
24     DDRC  = DDRC | (1<<PC0);      // Configure pin PC0 in PORTC as output pin
25     PORTC = PORTC | ~(1<<PC0);    // Set Value of PIN 0 in PORTC to 1 at the beginning (LED OFF)
26
27     while(1)
28     {
29
30     }
31 }
32
```

SESSION TASK

Practical Task: LED Toggling via External Interrupt

Objective:

Control an LED state (Toggle) using a Push Button connected to one of the External Interrupt pins in ATmega32.

Hardware Connections:

- LED: Connected to PC0 with negative logic
- Push Button: Connected to INTO (Pin 16 / PD2).
- Button Configuration: Use an external Pull-down resistor.

Task Requirements:

- Configure INTO to trigger on a falling Edge (when the button is pressed).
- Write an ISR (Interrupt Service Routine) for INTO.
- Inside the ISR, write the code to Toggle the LED state.
- The Main Loop should remain empty or perform another independent task

HOME TASK

- extend traffic project :
add emergency interrupt button with led any tybe or logic will be ok

